Compile-Time Programming in C++

Theory, Edge Cases & Code Examples

- ✓ What is Compile-Time Programming?
- ✓ constexpr Evolution Across Standards
- ✓ constexpr vs consteval vs constinit
- ✓ Why Compile-Time Programming Matters in Autonomous Vehicles
- ✓ Compile-Time Feature Selection Guide

→ *Swipe to learn* →

Compile-Time Programming in C++

Theory

📌 1. What is Compile-Time Programming?

Compile-time programming enables computations, type checks, and logic to execute during compilation rather than at runtime.

Core Benefits:

- **Zero runtime overhead**: Results baked into binary
- **Early error detection**: Bugs caught before program runs
- **Deterministic performance**: No runtime variation
- **Type safety**: Strong compile-time guarantees

Primary Tools:

Tool	C++ Version	Purpose	Evaluation
constexpr	C++11+	Functions/variables that *can* be compile-time	Optional
consteval	C++20	Functions that *must* be compile-time	Mandatory
static_assert	C++11	Compile-time assertions	Compile-time only
type_traits	C++11	Type introspection	Compile-time only
if constexpr	C++17	Conditional branching	Compile-time only
Templates	C++98+	Turing-complete metaprogramming	Compile-time only

---

Compile-Time Programming in C++

Theory

📌 2. constexpr Evolution Across Standards (1/2)

The `constexpr` keyword has evolved dramatically across C++ versions:

C++ Version	Restrictions	Example Operations
C++11	- Single return statement only - No loops - No mutable variables - No runtime deferral	`return (x > 0) ? x : -x;`

C++14	- Multiple statements allowed
	- Loops permitted
	- Local variables allowed
	- Can defer to runtime
		`int sum = 0;`
		`for(int i=0; i<n; ++i) sum += i;`
		`return sum;`

C++17	+ Lambda expressions + `if constexpr` branching	`if constexpr(cond) { ... }`
C++20	+ Virtual functions + try-catch + consteval functions	`consteval int foo() { }`

📌 2. constexpr Evolution Across Standards (2/2)

Example: Same Function Across Standards

```
// ❌ C++11: Too complex (no loops, no mutable variables)
// ✅ C++14+: Allowed
constexpr int sum_range(int n) {
    int total = 0;              // Mutable local variable
    for (int i = 1; i <= n; ++i) {  // Loop
        total += i;
    }
    return total;
}

// ✅ C++11: Recursive workaround
constexpr int sum_range_c11(int n) {
    return (n <= 0) ? 0 : n + sum_range_c11(n - 1);
}
```

Compile-Time Programming in C++

Theory

📌 3. constexpr vs consteval vs constinit

Feature	constexpr	consteval	constinit
C++ Version	C++11	C++20	C++20
Compile-time eval	Optional (if inputs are constant)	Mandatory (always)	Only for initialization
Runtime eval	✅ Allowed with runtime inputs	❌ Forbidden	N/A (for statics)
Use case	Flexible functions/variables	Strict compile-time functions	Static/thread_local variables
Context	Functions, variables, constructors	Functions only	Variables only

Code Examples:

```
// constexpr: Can run at compile-time OR runtime
constexpr int square(int x) {
    return x * x;
}

// consteval: MUST run at compile-time
consteval int square_compile_only(int x) {
    return x * x;
}

// constinit: Ensures compile-time initialization
constinit int global_config = 42;

int main() {
    constexpr int a = square(5);        // ✅ Compile-time
    int runtime_val = 10;
    int b = square(runtime_val);        // ✅ Runtime evaluation

    // int c = square_compile_only(runtime_val);  // ❌ Error: runtime value
    constexpr int d = square_compile_only(5);    // ✅ Compile-time

    global_config++;  // ✅ Can modify at runtime
}
```

📌 4. Why Compile-Time Programming Matters in Autonomous Vehicles

In safety-critical autonomous vehicle systems, compile-time guarantees prevent catastrophic failures:

Benefit	Impact	Autonomous Vehicle Example
Zero runtime overhead	Computations baked into binary	Lookup tables for sine/cosine in control loops
Early error detection	Bugs caught during compilation	Matrix dimension mismatches in sensor fusion
Deterministic timing	Zero timing variation	Control algorithms with hard 100Hz deadlines
Configuration validation	Invalid configs rejected at compile-time	Sensor count, calibration matrix sizes
Type safety	Type mismatches impossible at runtime	Type-specific sensor processing (LiDAR vs Radar)

Real-World Example: Camera Calibration Validation

```
// ❌ Runtime validation: Crashes in production if wrong dimensions
class CalibrationMatrix_Runtime {
    float data[12];  // No compile-time size check
public:
    CalibrationMatrix_Runtime(size_t rows, size_t cols) {
        assert(rows == 3 && cols == 4);  // Runtime check!
    }
};

// ✅ Compile-time validation: Impossible to create invalid matrix
template<size_t Rows, size_t Cols>
class CalibrationMatrix {
    static_assert(Rows == 3 && Cols == 4,
                  "Camera calibration must be 3x4 projection matrix");
    float data[Rows][Cols];
};

// Compile-time enforcement
CalibrationMatrix<3, 4> cam_matrix;        // ✅ Valid
// CalibrationMatrix<4, 4> invalid_matrix; // ❌ Compile error
```

📌 5. Compile-Time Feature Selection Guide

Use this decision matrix to choose the right compile-time feature:

Scenario	Best Tool	Reason
Mathematical constants (π, e)	`constexpr` variable	Zero runtime cost, clear intent
Configuration validation	`static_assert` + type traits	Compilation fails for invalid configs
Type-dependent algorithms	`if constexpr`	Branch elimination, zero overhead
Flexible computation	`constexpr` function	Can fall back to runtime if needed
Strict compile-time only	`consteval`	No runtime escape, guaranteed
Static init order safety	`constinit`	Prevents static initialization fiasco
Type-based computation	Template metaprogramming	Turing-complete at compile-time
Type checking/constraints	Type traits + `static_assert`	Compile-time type safety

Performance Characteristics:

Operation	Compilation Time	Binary Size	Runtime Cost
`constexpr` (compile-time)	High	Small increase	**Zero**
`constexpr` (runtime)	Low	Function code	Normal
Template metaprogramming	Very high (deep recursion)	Minimal	**Zero**
`if constexpr`	Low	Smaller (dead code removed)	**Zero**

Compile-Time Programming in C++

Edge Cases

📌 1. constexpr Runtime Deferral in C++11 vs C++14+ (1/2)

The behavior of `constexpr` functions with runtime arguments changed dramatically between C++11 and C++14.

```
constexpr int multiply(int x, int y) {
    return x * y;
}

int main() {
    int runtime_val = 5;

    // Scenario 1: Runtime call
    int result = multiply(runtime_val, 3);

    // C++11: ❌ Compile error (constexpr functions cannot be called
    //           with runtime values in non-constexpr context)
    // C++14+: ✅ Compiles and runs at runtime

    // Scenario 2: Compile-time call
    constexpr int compile_result = multiply(10, 20);
    // ✅ Works in all standards (evaluated at compile time)
}
```

Why the difference?

In C++11, `constexpr` functions were intended *only* for compile-time contexts. The language specification required that `constexpr` functions be usable as constant expressions, but didn't allow them to be regular functions. C++14 relaxed this, making `constexpr` mean "can be evaluated at compile time *if possible*".

Real-world implication (autonomous driving):

```
// C++14+ allows flexible compile-time/runtime switching
constexpr float deg_to_rad(float degrees) {
    return degrees * 3.14159f / 180.0f;
}

// Compile-time: steering angle limits baked into binary
constexpr float MAX_STEERING_RAD = deg_to_rad(30.0f);

// Runtime: dynamic angle conversion
float current_angle_deg = get_steering_angle();
float current_angle_rad = deg_to_rad(current_angle_deg);  // Runtime call
```

Compile-Time Programming in C++

Edge Cases

📌 1. constexpr Runtime Deferral in C++11 vs C++14+ (2/2)

Key takeaway: In C++14+, `constexpr` functions are dual-purpose (compile-time or runtime), but in C++11 they're compile-time only.

Compile-Time Programming in C++

Edge Cases

📌 2. constexpr with Loops and Mutable Variables (C++11 Restrictions) (1/2)

C++11 `constexpr` functions have severe restrictions that make many intuitive implementations invalid.

```
// ❌ C++11: Compile error (loops not allowed)
// ✅ C++14+: Valid
constexpr int sum_range(int n) {
    int total = 0;  // ❌ C++11: mutable local variable
    for (int i = 1; i <= n; ++i) {  // ❌ C++11: loop
        total += i;
    }
    return total;
}

// ✅ C++11: Valid (single return, ternary operator)
constexpr int sum_range_c11(int n) {
    return n <= 0 ? 0 : n + sum_range_c11(n - 1);
}

int main() {
    constexpr int result = sum_range(10);  // Requires C++14+
    constexpr int result_c11 = sum_range_c11(10);  // Works in C++11
}
```

Why C++11 is so restrictive?

The original design philosophy was that `constexpr` functions should be "pure" expressions without side effects. This made compiler implementation simpler but severely limited expressiveness. C++14 recognized this was too restrictive for practical use.

Workaround for C++11 (recursive approach):

```
// C++11-compatible recursive sum
constexpr int sum_recursive(int n) {
    return (n <= 1) ? n : n + sum_recursive(n - 1);
}

// But watch out for template instantiation depth limits!
constexpr int huge_sum = sum_recursive(10000);  // May hit recursion limit
```

📌 2. constexpr with Loops and Mutable Variables (C++11 Restrictions) (2/2)

Key takeaway: C++11 `constexpr` functions must be expressed as single-expression recursions; C++14+ allows imperative-style loops.

Compile-Time Programming in C++

Edge Cases

📌 3. if constexpr vs Regular if in Templates (1/2)

`if constexpr` (C++17) performs **compile-time branch elimination**, which is fundamentally different from runtime `if`.

```
#include <type_traits>
#include <iostream>

template<typename T>
void process(T value) {
    // Version 1: Regular if (runtime check)
    if (std::is_integral<T>::value) {
        // ❌ PROBLEM: This branch is instantiated even for non-integral types
        // If this code is invalid for T=float, compilation fails!
        value++;  // Would fail for std::string
        std::cout << "Integral: " << value << "\n";
    } else {
        std::cout << "Non-integral\n";
    }
}

template<typename T>
void process_constexpr(T value) {
    // Version 2: if constexpr (compile-time elimination)
    if constexpr (std::is_integral<T>::value) {
        // ✅ This branch is ONLY instantiated for integral types
        value++;
        std::cout << "Integral: " << value << "\n";
    } else {
        // ✅ This branch is ONLY instantiated for non-integral types
        std::cout << "Non-integral\n";
    }
}

int main() {
    process_constexpr(42);           // ✅ "Integral: 43"
    process_constexpr(3.14);         // ✅ "Non-integral"
    process_constexpr(std::string("test"));  // ✅ "Non-integral"

    // process(std::string("test"));  // ❌ Would fail: can't ++ a string
}
```

Compile-Time Programming in C++

Edge Cases

📌 3. if constexpr vs Regular if in Templates (2/2)

Why this matters:

Without `if constexpr`, **both branches** of the `if` must be syntactically valid for the instantiated type, even though only one runs. With `if constexpr`, the discarded branch is **never instantiated**, so it can contain code that wouldn't compile for that type.

Real-world scenario (sensor data processing):

```
template<typename SensorData>
void calibrate(SensorData& data) {
    if constexpr (std::is_same_v<SensorData, LidarPoint>) {
        // LiDAR-specific calibration (angle correction, intensity scaling)
        data.angle += angle_offset;
        data.intensity *= intensity_gain;
    } else if constexpr (std::is_same_v<SensorData, RadarReturn>) {
        // Radar-specific calibration (doppler correction)
        data.doppler_shift += doppler_offset;
    } else {
        // Generic calibration
        data.timestamp = adjust_timestamp(data.timestamp);
    }
}
```

Key takeaway: Use `if constexpr` for type-dependent code paths in templates; regular `if` checks at runtime and instantiates all branches.

Compile-Time Programming in C++

Edge Cases

📌 4. Template Instantiation Depth Limits (1/3)

Recursive template metaprogramming can hit compiler-imposed depth limits, causing compilation to fail.

```
// Compile-time factorial using template recursion
template<int N>
struct Factorial {
    static constexpr int value = N * Factorial<N - 1>::value;
};

template<>
struct Factorial<0> {
    static constexpr int value = 1;
};

int main() {
    constexpr int f5 = Factorial<5>::value;      // ✅ 120
    constexpr int f10 = Factorial<10>::value;    // ✅ 3628800

    // ❌ Most compilers fail: exceeds default depth limit (~512-1024)
    // constexpr int f2000 = Factorial<2000>::value;
    // Error: template instantiation depth exceeds maximum of 1024
}
```

Compile-Time Programming in C++

Edge Cases

📌 4. Template Instantiation Depth Limits (2/3)

Why this happens:

Template metaprogramming works by recursive template instantiation. The compiler must generate `Factorial<N>`, which requires `Factorial<N-1>`, which requires `Factorial<N-2>`, and so on. This creates a chain of 2000 instantiations, exceeding compiler limits.

Solutions:

1. **Increase compiler limit**:
   `bash g++ -ftemplate-depth=5000 program.cpp clang++ -ftemplate-depth=5000 program.cpp`
2. **Use constexpr functions instead** (C++14+):
   ```cpp
   // ✅ No instantiation depth limit (regular recursion at compile time)
   constexpr int factorial_constexpr(int n) {
   return (n <= 1) ? 1 : n * factorial_constexpr(n - 1);
   }
   constexpr int f2000 = factorial_constexpr(2000); // ✅ Works (huge number)
   ```

Why constexpr doesn't hit the limit:

`constexpr` functions use regular function call recursion (evaluated at compile time), not template instantiation. The recursion limit is typically much higher (stack depth limit), not a hard-coded compiler constant.

Real-world consideration:

```
// Computing polynomial coefficients at compile time
template<int Degree>
struct PolynomialCoefficients {
    // Recursive computation...
    // If Degree = 1000, template instantiation may fail
};

// ✅ Better: Use constexpr function
constexpr std::array<double, N> compute_coefficients() {
    // Iterative or recursive computation
}
```

📌 4. Template Instantiation Depth Limits (3/3)

Key takeaway: For deep recursion (>100 levels), prefer `constexpr` functions over template metaprogramming to avoid instantiation depth limits.

Compile-Time Programming in C++

Edge Cases

📌 5. consteval Forces Compile-Time Evaluation (C++20) (1/2)

`consteval` (immediate functions) **must** produce a constant expression and cannot be called with runtime values.

```
// consteval: MUST be compile-time evaluated
consteval int square(int x) {
    return x * x;
}

// constexpr: CAN be compile-time evaluated
constexpr int cube(int x) {
    return x * x * x;
}

int main() {
    // ✅ Compile-time: both work
    constexpr int sq = square(5);   // 25
    constexpr int cb = cube(5);     // 125

    // Runtime value
    int runtime_val = 10;

    // ❌ Compile error: consteval cannot accept runtime argument
    // int result = square(runtime_val);

    // ✅ constexpr can accept runtime argument (evaluated at runtime)
    int result = cube(runtime_val);  // Runs at runtime

    // ✅ Workaround: force compile-time evaluation
    constexpr int val = 10;
    int result2 = square(val);  // OK: val is constexpr
}
```

Compile-Time Programming in C++

Edge Cases

📌 5. consteval Forces Compile-Time Evaluation (C++20) (2/2)

Use cases for consteval:

1. **Configuration validation** that should never happen at runtime:
   ```cpp
   consteval bool validate_sensor_config(int sensor_count, int sample_rate) {
   if (sensor_count < 1 || sensor_count > 16) return false;
   if (sample_rate < 100 || sample_rate > 10000) return false;
   return true;
   }
   ```
   ```cpp
   // ✅ Validated at compile time
   static_assert(validate_sensor_config(8, 1000), "Invalid sensor config");

   // ❌ Cannot validate runtime values
   // int count = get_sensor_count();
   // validate_sensor_config(count, 1000); // Compile error
   ```

1. **Embedded systems** where runtime computation is forbidden:
   ```cpp
   consteval size_t compute_buffer_size(size_t sensors, size_t samples) {
   return sensors * samples * sizeof(float);
   }
   ```
   ```cpp
   // Buffer size computed at compile time (no runtime overhead)
   alignas(64) char buffer[compute_buffer_size(8, 1024)];
   ```

Key takeaway: Use `consteval` when you want to guarantee compile-time evaluation and prevent accidental runtime usage.

Compile-Time Programming in C++

Code Examples

📌 1. Basic constexpr Function (Easy Level) (1/2)

This basic example demonstrates:
- **constexpr functions** that work in both compile-time and runtime contexts
- **constexpr variables** as named constants
- **static_assert** for compile-time validation
- **Recursive constexpr** functions (single-expression recursion for C++11 compatibility)

The factorial function is evaluated at compile time when `fact5` is initialized (because it's `constexpr`), but evaluated at runtime for `fact7` (because `n` is not a constant expression).

```
#include <iostream>

// Simple constexpr function (works in C++11+)
constexpr int factorial(int n) {
    return (n <= 1) ? 1 : n * factorial(n - 1);
}

// constexpr variable
constexpr int MAX_CONNECTIONS = 10;

// constexpr with static_assert
constexpr int compute_buffer_size(int elements) {
    return elements * sizeof(int);
}

int main() {
    // Compile-time evaluation
    constexpr int fact5 = factorial(5);
    std::cout << "5! = " << fact5 << "\n";  // 120

    // Can also use at runtime (C++14+, but this simple version works in C++11)
    int n = 7;
    int fact7 = factorial(n);  // Evaluated at runtime
    std::cout << "7! = " << fact7 << "\n";  // 5040

    // Compile-time buffer size calculation
    constexpr int buffer_size = compute_buffer_size(100);
    char buffer[buffer_size];  // Array size must be compile-time constant

// ...continues on next slide
```

Compile-Time Programming in C++

Code Examples

📌 1. Basic constexpr Function (Easy Level) (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    // Compile-time assertion
    static_assert(factorial(3) == 6, "Factorial calculation wrong");
    static_assert(MAX_CONNECTIONS > 0, "Must have at least 1 connection");

    return 0;
}
```

Compile-Time Programming in C++

Code Examples

📌 2. constexpr with Loops and Branches (C++14 Features) (1/3)

This C++14 example shows:
- **Loops in constexpr functions** (not allowed in C++11)
- **Mutable local variables** in constexpr functions
- **Multiple return statements** and complex control flow
- **Real-world use case**: Compile-time CRC32 hashing for efficient string comparisons

The CRC32 example is particularly powerful: string hashes are computed at compile time, allowing switch-case on strings (which normally isn't possible in C++).

```cpp
#include <iostream>

// C++14: constexpr with loops and mutable variables
constexpr int sum_range(int n) {
    int total = 0;
    for (int i = 1; i <= n; ++i) {
        total += i;
    }
    return total;
}

// C++14: constexpr with multiple return statements and conditionals
constexpr int abs_value(int x) {
    if (x < 0) return -x;
    return x;
}

// C++14: constexpr with complex control flow
constexpr int fibonacci(int n) {
    if (n <= 1) return n;

    int a = 0, b = 1;
    for (int i = 2; i <= n; ++i) {
        int temp = a + b;
        a = b;
        b = temp;
    }
    return b;
}
// ...continues on next slide
```

Compile-Time Programming in C++

Code Examples

📌 2. constexpr with Loops and Branches (C++14 Features) (2/3)

```
// ...continued from previous slide

// Real-world: Compile-time CRC table generation
constexpr unsigned int crc32_byte(unsigned int crc, unsigned char byte) {
    crc ^= byte;
    for (int i = 0; i < 8; ++i) {
        crc = (crc >> 1) ^ (0xEDB88320 & -(crc & 1));
    }
    return crc;
}

constexpr unsigned int crc32_string(const char* str, size_t len) {
    unsigned int crc = 0xFFFFFFFF;
    for (size_t i = 0; i < len; ++i) {
        crc = crc32_byte(crc, str[i]);
    }
    return ~crc;
}

int main() {
    // Compile-time evaluation
    constexpr int sum = sum_range(100);          // Sum 1-100 = 5050
    constexpr int fib10 = fibonacci(10);         // 55
    constexpr int abs_neg = abs_value(-42);      // 42

    std::cout << "Sum 1-100: " << sum << "\n";
    std::cout << "Fib(10): " << fib10 << "\n";
    std::cout << "abs(-42): " << abs_neg << "\n";

    // Compile-time string hashing for switch-case on strings
    constexpr unsigned int sensor_type = crc32_string("LIDAR", 5);

    // Can use compile-time hash in switch
    unsigned int incoming_sensor = get_sensor_type_hash();
    switch (incoming_sensor) {
        case crc32_string("LIDAR", 5):
            process_lidar();
            break;
        case crc32_string("RADAR", 5):
            process_radar();
            break;
// ...continues on next slide
```

Compile-Time Programming in C++

Code Examples

📌 2. constexpr with Loops and Branches (C++14 Features) (3/3)

[Code continues across slides]

```
// ...continued from previous slide
        case crc32_string("CAMERA", 6):
            process_camera();
            break;
    }

    return 0;
}
```

Compile-Time Programming in C++

Code Examples

📌 3. static_assert with type_traits (Type Checking) (1/3)

This example demonstrates:
- **static_assert with type_traits** for compile-time type validation
- **Template constraints** enforced at compile time
- **Dimension checking** for matrix multiplication (impossible dimension mismatches caught at compile time)
- **Real-world safety**: In autonomous vehicles, matrix dimension errors can cause catastrophic failures; catching them at compile time is critical

---

```
#include <type_traits>
#include <iostream>

// Template function with compile-time type checking
template<typename T>
void process_numeric(T value) {
    // ✅ Ensure T is arithmetic (int, float, double, etc.)
    static_assert(std::is_arithmetic<T>::value,
                  "T must be an arithmetic type");

    std::cout << "Processing: " << value * 2 << "\n";
}

// Template class with size constraints
template<typename T, size_t Size>
class FixedArray {
    static_assert(Size > 0, "Array size must be greater than 0");
    static_assert(Size <= 1024, "Array size too large (max 1024)");
    static_assert(std::is_trivially_copyable<T>::value,
                  "T must be trivially copyable for memcpy optimization");

    T data[Size];

public:
    constexpr size_t size() const { return Size; }
    T& operator[](size_t idx) { return data[idx]; }
};

// Autonomous vehicle: Compile-time matrix dimension validation
template<size_t Rows, size_t Cols>
// ...continues on next slide
```

Compile-Time Programming in C++

Code Examples

📌 3. static_assert with type_traits (Type Checking) (2/3)

```
// ...continued from previous slide
class Matrix {
    static_assert(Rows > 0 && Cols > 0, "Matrix dimensions must be positive");

    float data[Rows][Cols];

public:
    static constexpr size_t rows() { return Rows; }
    static constexpr size_t cols() { return Cols; }

    // Only allow multiplication if dimensions match
    template<size_t OtherCols>
    Matrix<Rows, OtherCols> multiply(const Matrix<Cols, OtherCols>& other) const {
        // Inner dimensions must match (this->Cols == other->Rows)
        // This is guaranteed by the template parameter Cols
        Matrix<Rows, OtherCols> result{};
        for (size_t i = 0; i < Rows; ++i) {
            for (size_t j = 0; j < OtherCols; ++j) {
                for (size_t k = 0; k < Cols; ++k) {
                    result.data[i][j] += data[i][k] * other.data[k][j];
                }
            }
        }
        return result;
    }
};

int main() {
    // ✅ Valid: int is arithmetic
    process_numeric(42);

    // ❌ Compile error: std::string is not arithmetic
    // process_numeric(std::string("hello"));

    // ✅ Valid: size within limits
    FixedArray<int, 100> arr;

    // ❌ Compile error: size is 0
    // FixedArray<int, 0> invalid_arr;

// ...continues on next slide
```

Compile-Time Programming in C++

Code Examples

📌 3. static_assert with type_traits (Type Checking) (3/3)

[Code continues across slides]

```
// ...continued from previous slide
    // ✅ Valid matrix operations (dimensions checked at compile time)
    Matrix<3, 4> projection;   // 3x4 projection matrix
    Matrix<4, 4> transform;    // 4x4 transformation matrix

    // ✅ Valid: 3x4 * 4x4 = 3x4
    auto result = projection.multiply(transform);

    // ❌ Compile error: dimension mismatch (3x4 * 3x4 invalid)
    // auto invalid = projection.multiply(projection);

    return 0;
}
```

Compile-Time Programming in C++

Code Examples

📌 4. if constexpr for Type-Dependent Code (C++17) (1/3)

This example shows:
- **if constexpr for type-dependent code paths** (branches not taken are never instantiated)
- **Multiple type checks** with `is_same_v`, `is_arithmetic_v`, `is_pointer_v`
- **Real-world sensor processing** with type-specific algorithms
- **Compile-time branch elimination** for zero runtime overhead

The key advantage: Each instantiation only contains the code path for that specific type. `serialize<int>` doesn't contain string handling code, and `serialize<std::string>` doesn't contain arithmetic handling code.

```
#include <iostream>
#include <type_traits>
#include <string>
#include <vector>

// Generic serialization with type-dependent logic
template<typename T>
void serialize(const T& value) {
    if constexpr (std::is_arithmetic_v<T>) {
        // Arithmetic types: write binary
        std::cout << "Binary serialize: " << value << "\n";
    } else if constexpr (std::is_same_v<T, std::string>) {
        // Strings: write length + data
        std::cout << "String serialize: " << value.size() << " bytes\n";
    } else if constexpr (std::is_pointer_v<T>) {
        // Pointers: serialize pointed-to value
        std::cout << "Pointer serialize: " << *value << "\n";
    } else {
        // Unknown types: static assertion failure
        static_assert(std::is_arithmetic_v<T>, "Unsupported type for serialization");
    }
}

// Autonomous vehicle: Type-specific sensor data processing
template<typename SensorData>
class SensorProcessor {
public:
    void process(const SensorData& data) {
        if constexpr (std::is_same_v<SensorData, LidarPoint>) {
// ...continues on next slide
```

Compile-Time Programming in C++

Code Examples

📌 4. if constexpr for Type-Dependent Code (C++17) (2/3)

```
// ...continued from previous slide
            // LiDAR-specific processing
            process_lidar(data);
        } else if constexpr (std::is_same_v<SensorData, RadarReturn>) {
            // Radar-specific processing
            process_radar(data);
        } else if constexpr (std::is_same_v<SensorData, CameraFrame>) {
            // Camera-specific processing
            process_camera(data);
        } else {
            static_assert(always_false<SensorData>::value,
                          "Unknown sensor type");
        }
    }

private:
    template<typename T>
    struct always_false : std::false_type {};

    void process_lidar(const LidarPoint& point) {
        // Distance filtering, ground removal, clustering
        if (point.distance < 100.0f && point.z > -2.0f) {
            // Valid point for obstacle detection
        }
    }

    void process_radar(const RadarReturn& ret) {
        // Doppler velocity extraction, range-rate filtering
        if (ret.doppler_velocity > 0.0f) {
            // Approaching object
        }
    }

    void process_camera(const CameraFrame& frame) {
        // Object detection, lane marking extraction
    }
};

// Template function that adapts behavior based on container type
template<typename Container>
void print_container(const Container& c) {
// ...continues on next slide
```

Compile-Time Programming in C++

Code Examples

📌 4. if constexpr for Type-Dependent Code (C++17) (3/3)

[Code continues across slides]

```
// ...continued from previous slide
    if constexpr (std::is_same_v<Container, std::string>) {
        // Special handling for strings
        std::cout << "String: \"" << c << "\"\n";
    } else {
        // Generic container iteration
        std::cout << "Container: [";
        bool first = true;
        for (const auto& elem : c) {
            if (!first) std::cout << ", ";
            std::cout << elem;
            first = false;
        }
        std::cout << "]\n";
    }
}

int main() {
    // Arithmetic serialization
    serialize(42);
    serialize(3.14);

    // String serialization
    serialize(std::string("hello"));

    // Pointer serialization
    int value = 100;
    serialize(&value);

    // Container printing with type adaptation
    print_container(std::vector<int>{1, 2, 3, 4, 5});
    print_container(std::string("hello world"));

    return 0;
}
```

Compile-Time Programming in C++

Code Examples

📌 5. Template Metaprogramming (Advanced) (1/4)

This advanced example demonstrates:
- **Template recursion** for compile-time computation
- **Template specialization** for base cases
- **Type lists** and metaprogramming abstractions
- **Variadic templates** with fold expressions (C++17)
- **Compile-time lookup tables** for runtime efficiency

Template metaprogramming is Turing-complete: you can compute anything at compile time that you could compute at runtime, but with zero runtime overhead.

```
#include <iostream>

// Classic: Compile-time factorial using template recursion
template<int N>
struct Factorial {
    static constexpr int value = N * Factorial<N - 1>::value;
};

template<>
struct Factorial<0> {
    static constexpr int value = 1;
};

// Compile-time power calculation
template<int Base, int Exponent>
struct Power {
    static constexpr int value = Base * Power<Base, Exponent - 1>::value;
};

template<int Base>
struct Power<Base, 0> {
    static constexpr int value = 1;
};

// Compile-time Fibonacci
template<int N>
struct Fibonacci {
// ...continues on next slide
```

Compile-Time Programming in C++

Code Examples

📌 5. Template Metaprogramming (Advanced) (2/4)

```
// ...continued from previous slide
    static constexpr int value = Fibonacci<N - 1>::value + Fibonacci<N - 2>::value;
};

template<>
struct Fibonacci<0> {
    static constexpr int value = 0;
};

template<>
struct Fibonacci<1> {
    static constexpr int value = 1;
};

// Type list (metaprogramming foundation)
template<typename... Types>
struct TypeList {};

// Length of type list
template<typename List>
struct Length;

template<typename... Types>
struct Length<TypeList<Types...>> {
    static constexpr size_t value = sizeof...(Types);
};

// Get type at index
template<size_t Index, typename List>
struct TypeAt;

template<size_t Index, typename Head, typename... Tail>
struct TypeAt<Index, TypeList<Head, Tail...>> {
    using type = typename TypeAt<Index - 1, TypeList<Tail...>>::type;
};

template<typename Head, typename... Tail>
struct TypeAt<0, TypeList<Head, Tail...>> {
    using type = Head;
};

// ...continues on next slide
```

Compile-Time Programming in C++

Code Examples

📌 5. Template Metaprogramming (Advanced) (3/4)

```
// ...continued from previous slide
// Check if type exists in list
template<typename T, typename List>
struct Contains;

template<typename T, typename... Types>
struct Contains<T, TypeList<Types...>> {
    static constexpr bool value = (std::is_same_v<T, Types> || ...);
};

// Real-world: Compile-time lookup table generation
template<size_t Size>
struct SineLookupTable {
    static constexpr double PI = 3.14159265358979323846;

    double data[Size];

    constexpr SineLookupTable() : data{} {
        for (size_t i = 0; i < Size; ++i) {
            data[i] = compute_sin(2.0 * PI * i / Size);
        }
    }

private:
    // Taylor series approximation for sine (compile-time)
    static constexpr double compute_sin(double x) {
        double result = x;
        double term = x;
        for (int n = 1; n < 10; ++n) {
            term *= -x * x / ((2 * n) * (2 * n + 1));
            result += term;
        }
        return result;
    }
};

int main() {
    // Template metaprogramming results
    constexpr int fact5 = Factorial<5>::value;
    constexpr int pow2_10 = Power<2, 10>::value;
    constexpr int fib10 = Fibonacci<10>::value;
// ...continues on next slide
```

Compile-Time Programming in C++

Code Examples

📌 5. Template Metaprogramming (Advanced) (4/4)

[Code continues across slides]

```
// ...continued from previous slide

    std::cout << "5! = " << fact5 << "\n";        // 120
    std::cout << "2^10 = " << pow2_10 << "\n";    // 1024
    std::cout << "Fib(10) = " << fib10 << "\n";   // 55

    // Type list operations
    using SensorTypes = TypeList<LidarPoint, RadarReturn, CameraFrame>;
    constexpr size_t num_sensors = Length<SensorTypes>::value;
    std::cout << "Number of sensor types: " << num_sensors << "\n";  // 3

    // Check if type exists
    constexpr bool has_lidar = Contains<LidarPoint, SensorTypes>::value;
    constexpr bool has_gps = Contains<GpsCoordinate, SensorTypes>::value;
    std::cout << "Has LiDAR: " << has_lidar << "\n";  // true
    std::cout << "Has GPS: " << has_gps << "\n";      // false

    // Compile-time sine table (256 entries computed at compile time)
    constexpr SineLookupTable<256> sine_table;
    std::cout << "sin(π/2) ≈ " << sine_table.data[64] << "\n";  // ~1.0

    return 0;
}
```

What You Learned

Key Takeaways

- ✓ What is Compile-Time Programming?
- ✓ constexpr Evolution Across Standards
- ✓ constexpr vs consteval vs constinit
- ✓ Why Compile-Time Programming Matters in Autonomous Vehicles

→ *Swipe to learn* →